HARDWARE IMPLEMENTATION OF MODULES OF DUAL BRANCH CNN ARCHITECTURE

MINI PROJECT REPORT

Submitted by

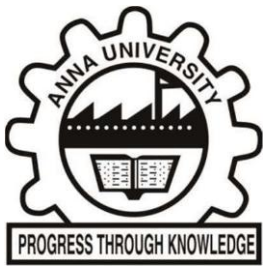
ESHAANJANA S – 2023105011
MEGANATH VISHWA M - 2023105033
KANNAN S A - 2023105538
VINOTHINI K - 2023105552

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING



COLLEGE OF ENGINEERING GUINDY

ANNA UNIVERSITY: CHENNAI 600025

JULY 2026 - NOVEMBER 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this report titled **"HARDWARE IMPLEMENTATION OF MODULES OF DUAL BRANCH 2D CNN ARCHITECTURE"** is the Bonafide work of ESHAANJANA S (2023105011), MEGANATH VISHWA M (2023105033), KANNAN S A (2023105538), VINOTHINI K (2023105552)**,** who carried out the project work under my supervision.

SIGNATURE
Dr. M.A Bhagyaveni
HEAD OF THE DEPARTMENT
Professor
Department of ECE
College of Engineering Guindy
Anna University
Chennai-600025

SIGNATURE
Dr. C.M Sujatha
SUPERVISOR
Professor
Department of ECE
College of Engineering Guindy
Anna University
Chennai-600025

ACKNOWLEDGEMENT

We express our sincere gratitude to our Dean, **Dr. S. Valli**, College of Engineering, Guindy, for her valuable support and encouragement throughout the course of this mini project.

We extend our heartfelt appreciation to our Head of the Department, **Dr. M. A. Bhagyaveni**, Professor, Department of Electronics and Communication Engineering, for her support and guidance throughout the project.

We are immensely grateful to our mini-project supervisor, **Dr.C.M Sujatha**, for her valuable guidance, technical mentorship, patience, and continuous encouragement throughout the development of this project, titled *"**Hardware Implementation of Modules of Dual Branch 2D CNN Architecture**."* Her insightful suggestions and support have been instrumental in the successful progress of this work.

We also express our sincere thanks to all the teaching and non-teaching staff of the Department of Electronics and Communication Engineering for their valuable assistance, cooperation, and support during this mini project.

சுருக்கம்

களத்தில் நிரலமைக்கக்கூடிய மின்னணு வாயில் வரிசைகள், இணைச் செயலாக்கம், மறுகட்டமைப்புத் திறன் மற்றும் வன்பொருள் வளங்களைத் திறமையாகப் பயன்படுத்துதல் போன்ற நன்மைகளை வழங்குவதால், அதிகக் கணக்கீட்டுச் சிக்கலுள்ள ஆழக் கற்றல் கட்டமைப்புகளைச் செயல்படுத்துவதற்கு ஏற்றவையாக உள்ளன. வழக்கமான சுருள்வடிவ நரம்பியல் வலையமைப்புகள் அதிக எண்ணிக்கையிலான பெருக்கல்–குவித்தல் செயல்பாடுகளையும் நினைவக அணுகல்களையும் தேவைப்படுத்துவதால், கணக்கீட்டுச் சிக்கலும் வன்பொருள் வளப் பயன்பாடும் அதிகரிக்கின்றன. இச்சவால்களைக் குறைப்பதற்காக, இந்தத் திட்டம் இரட்டைக் கிளைக் கட்டமைப்பிற்கான மூன்று தனிப்பயன் வன்பொருள் தொகுதிகளை முன்மொழிகிறது. அவை, இணைச் செயலாக்கத்தின் மூலம் சுருள்வடிவக் கணக்கீடுகளை மேற்கொள்ளும் தொகுதி, பூஜ்ஜிய மதிப்புடைய வடிகட்டி எடைகளுக்கான தேவையற்ற கணக்கீடுகளைத் தவிர்க்கும் தொகுதி, மற்றும் கூட்டல், பெருமதிப்புத் தேர்வு ஆகிய செயல்பாடுகளை ஒருங்கிணைக்கும் தொகுதி ஆகியவையாகும். தரவுப் பிரதிநிதித்துவத் தேவைகளையும் வன்பொருள் சிக்கலையும் குறைப்பதற்காக, இந்தக் கட்டமைப்பு நான்கு பிட் அளவாக்கப்பட்ட உள்ளீட்டுப் படத்துணுக்குகளைப் பயன்படுத்துகிறது. இத்தொகுதிகள் பதிவேற்ற நிலை வடிவமைப்பு முறையில் உருவாக்கப்பட்டு, வன்பொருள் உருவகப்படுத்தல் மற்றும் தொகுப்புக் கருவிகள் மூலம் மதிப்பீடு செய்யப்பட்டு, மென்பொருள் மூலம் நிர்வகிக்கப்படும் வன்பொருள் கட்டமைப்பு வழியாக நிரலமைக்கக்கூடிய மின்னணு வாயில் வரிசைத் தளத்தில் ஒருங்கிணைக்கப்படுகின்றன. இந்தச் செயலாக்கத்தின் நோக்கம், செயலாக்கத் தாமதத்தைக் குறைத்து, செயல்திறனையும் வன்பொருள் வளப் பயன்பாட்டுத் திறனையும் மேம்படுத்தி, நோய் கண்டறிதல் மற்றும் வகைப்படுத்தலுக்கான தன்னிறைவான விளிம்புக் கணினிச் செயலாக்க அமைப்பை உருவாக்குவதாகும்.

ABSTRACT

Field-Programmable Gate Arrays (FPGAs) provide parallel processing, reconfigurability, and efficient hardware resource utilization, making them suitable for implementing computationally intensive deep learning architectures. However, conventional Convolutional Neural Networks (CNN) require numerous multiply-accumulate (MAC) operations and frequent memory accesses, increasing computational complexity and hardware resource consumption. This project presents the hardware implementation of three custom modules for a Dual-Branch CNN architecture: Parallel Quantized Pixel-wise (PQP) convolution, Skip-Zero-Weight (SZW), and a function-merging addition–pooling unit. The PQP module performs convolution operations using parallel processing and reduced-precision pixel data to improve computational efficiency. The SZW module skips multiplication and accumulation operations associated with zero-valued filter weights, thereby reducing unnecessary computations. The function-merging addition–pooling unit combines addition and pooling operations within a single processing module to minimize intermediate data storage and memory transfers. The architecture uses 4-bit quantized input pixels to reduce data representation requirements and hardware complexity. The custom modules are described using Register Transfer Level (RTL), evaluated through hardware simulation and synthesis tools, and integrated into the PYNQ-Z2 FPGA platform through a software-managed hardware overlay. The proposed implementation aims to reduce processing latency, improve throughput, and utilize FPGA resources efficiently, providing an entirely independent, standalone edge acceleration system.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT (TAMIL)	iv
	ABSTRACT (ENGLISH)	v
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
	LIST OF ABBREVIATIONS	x
1.	**INTRODUCTION**	1
	1.1 Project Overview	1
	1.2 Objectives	3
	1.3 Report Organization	3
2.	**LITERATURE REVIEW**	4
	2.1 Inferences from Literature review	6
3.	**METHODOLOGY**	7
	3.1 Methodology Overview	7
	3.2 Workflow of System Architecture	7
	3.3 Software and Hardware Requirements	9
	3.4 ECG Dataset Preparation and Pre - Processing	9
	3.5 Dual Branch CNN Architecture	11
	3.5.1 Skip Zero Weight (SZW) Unit Architecture	13

3.5.2 Parallel-Quantized-Pixel-wise (PQP) convolution architecture 15
3.5.3 Add-Pool Unit Architecture 17
3.6 Dual Branch CNN Model Development 18
3.7 PS - PL IP Design and Hardware Deployment 19

4. RESULTS AND DISCUSSION 21
4.1 Illustration of the Convolution Operation 21
4.2 Dual-Branch Convolution and Feature-Map Addition 23
4.3 Implementation of Max Pooling unit 24
4.4 Simulation Analysis of final layer 25
4.5 FPGA Resource Utilization and Evaluation 27
4.6 Software Model Training 29
4.6.1 Classification Performance 30
4.6.2 Classification Performance after INT4 Quantization 31

5. CONCLUSION 32
6. REFERENCES 33

LIST OF FIGURES

FIGURE NO	NAME	PAGE NO
3.1	Software Workflow	7
3.2	Representation of ECG signals for Arrythmia classification	11
3.3	Hardware Control Architecture	12
3.4	Skip Zero Weight Architecture	14
3.5	Architectural Flow of PQP Convolution Layers	15
3.6	PQP Layer II Architectural Flow	16
3.7	Add-Pool Architecture	17
3.8	Work flow of CNN training	18
3.9	PS – PL Integrated Architecture Design	19
4.1	Illustration of Convolution operation of ECG input image	21
4.2	RTL Simulation waveform of the final convolution layer	22
4.3	ECG Image Representation of Individual Branch Outputs	23
4.4	Representation of ECG signals before and after Max Pooling Unit	24
4.5	Outputs of Each Layer	25
4.6	Simulation Waveform of Final Layer of PQP Convolution	26
4.7	Resource Utilisation of PQP Convolution	27
4.8	Resource Utilisation of Standard Convolution	27
4.9	Experimental Setup	28
4.10	Performance Curve of trained model	29
	Confusion Matrix and Class-Wise	

4.11	Performance Metrics of the ECG Classification Model	30
4.12	Classification Performance After Quantisation	31

LIST OF TABLES

TABLE NO	NAME	PAGE NO
3.1	Dataset Details	9
3.2	Hyperparameters of each Module	13
4.1	Comparison of PQP and Standard Convolution	29
4.2	Summary of Performance with Different Activation Precision	31

LIST OF ABBREVIATIONS

AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
CNN	Convolutional Neural Network
DMA	Direct Memory Access
ECG	Electro Cardio Gram
FPGA	Field Programmable Gate Arrays
MAC	Multiply and Accumulation Control
MIT-BIH	Massachusetts Institute of Technology and Boston's Beth Israel Hospital.
PC	Personal Computer
PL	Programmable Logic
PQP	Parallel Quantised Pixel
PS	Processing System
PYNQ	Python Productivity for Zynq
RTL	Register Transfer Level
SZW	Skip Zero Weight

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

Convolutional Neural Networks (CNN) is widely used in deep learning models for image processing and classification. The main purpose of CNN is to automatically learn meaningful features from input images and use them to identify objects, recognize patterns, and distinguish between different categories without requiring manual feature extraction. It extracts important features such as edges, shapes, textures, and patterns, making it suitable for applications such as medical image analysis, facial recognition, object detection, and automated inspection. CNN consists of several layers, including convolutional layers, activation functions, pooling layers, and fully connected layers. The convolutional layer extracts features using kernel filters, while activation functions such as ReLU introduce non-linearity, enabling the network to learn complex patterns. Pooling reduces the feature map size and computational requirements while retaining important information. The fully connected layer uses the extracted features to perform classification or prediction. By combining these layers, CNNs progressively learn low-level features, such as edges, and high-level features, such as shapes and complex patterns, making them effective for processing image-based data.

FPGA provides advantages such as parallel processing, reconfigurability, low power consumption, and flexible hardware resource utilization, making them suitable for implementing CNNs. However, CNNs require extensive computations and memory accesses, particularly in convolution layers that perform numerous MAC operations to extract image features. These operations increase processing time and hardware resource usage. In addition, storing intermediate results requires extra memory and data transfers,

which can reduce overall system efficiency. Therefore, an optimized hardware architecture is required to perform computations in parallel, eliminate unnecessary operations, minimize intermediate data storage, and utilize FPGA resources efficiently.

This project implements a Dual-Branch 2D-CNN architecture to improve computational efficiency and exploit the parallel processing capabilities of an FPGA. ECG signals are converted into 2D images and processed using unsigned 4-bit integer quantization to reduce memory requirements and computational complexity. PQP convolution module performs multiple pixel-wise convolution operations in parallel, improving processing speed. The function-merging add-pool unit combines addition and pooling operations into a single hardware unit, reducing intermediate data storage and memory access. SZW architecture avoids unnecessary MAC operations when filter weights are zero and power consumption. The dual-branch CNN processes feature information through parallel paths to support effective feature extraction and classification. The complete RTL design is developed using Vivado which is then utilised for synthesis, implementation, and resource utilization analysis. The optimized design is packaged as a custom RTL IP core and integrated with Zynq ultrascale processing system and created block design. Then, the hybrid PS-PL block design undergoes synthesize, implementation, and RTL netlist generation. The RTL is converted into bitstream file to deploy in PYNQ Z2 board. The project primarily focuses on verifying the functionality of the three hardware modules, analyzing their outputs, and evaluating their effectiveness in improving hardware efficiency.

1.2 OBJECTIVES:

- To design and implement modules such as PQP,SZW and Add Pool units of Dual Branch 2D-CNN architecture at the Register Transfer Level (RTL) using Verilog.
- To eliminate standard ReLU overhead by leveraging unsigned 4-bit integer calculations, ensuring all intermediate results remain naturally non-negative.

1.3 REPORT ORGANIZATION:

This report is organized into four chapters. Chapter 1, Introduction, provides a detailed overview of the project, explains the problem statement, and describes the main objectives of implementing hardware optimized modules of a Dual Branch 2D CNN architecture on an FPGA platform. Chapter 2, Literature Review, discusses existing research on convolutional neural networks, FPGA based image processing, quantization techniques, and hardware optimization methods, highlighting the approaches used to improve computational efficiency and reduce hardware requirements. Chapter 3, Design and Implementation, explains the architecture and working principles of the proposed Dual Branch 2D CNN and describes the three main hardware modules, namely the Parallel Quantized Pixel convolution module, the function merging addition and pooling unit, and the Skip Zero Weight architecture. It also covers the RTL design, simulation process, and implementation of the designed modules on the PYNQ Z2 FPGA board. Chapter 4, Results and Discussion, presents the simulation outputs, hardware implementation results, FPGA resource utilization, processing performance, and a detailed analysis of the effectiveness of the implemented modules in improving computational efficiency and reducing hardware resource requirements.

CHAPTER 2

LITERATURE REVIEW

Huynh (2022) presented a hardware–software co-design approach for implementing convolution operations on CNN using the PYNQ-Z2 FPGA platform. The study developed a custom convolution IP core and integrated it with software overlays to improve computational performance. The implementation demonstrated the advantages of executing computationally intensive operations using dedicated hardware rather than relying entirely on software processing. This work provides a foundation for developing customized convolution modules on embedded FPGA platforms using parallel processing to improve computational efficiency.

Sreenivasulu and Meenpal (2019) proposed an efficient FPGA implementation of two-dimensional convolution for image-processing applications. Their approach employs pipelined shift registers and line buffers to store image rows and generate sliding windows for convolution. Instead of storing the complete image in large frame buffers, the architecture processes image data as it arrives, thereby reducing memory requirements and supporting continuous processing. The study emphasizes the importance of efficient data buffering and pipelining in FPGA-based image processing utilizing reduced intermediate data storage and memory transfers.

Li et al., (2020) proposed CNN architecture designed to improve hardware efficiency for real-time applications on FPGA platforms. The architecture combines pointwise convolution with conventional two-dimensional convolution and employs reduced-precision activations and binary weights to reduce computational

complexity. The use of simplified arithmetic operations and optimized convolution structures helps reduce hardware requirements and power consumption. This study demonstrates the importance of selecting suitable numerical representations and optimizing convolution operations for resource-constrained hardware.

Hailesellasie et al., (2018) proposed an FPGA-based CNN accelerator that reduces model parameters and computational complexity. The design uses smaller convolution kernels and reduces the number of fully connected layers, retaining only the final classification layer. These architectural modifications reduce the storage requirements and hardware resources needed for implementation. The architecture was implemented using Xilinx Vivado on the Zynq XC7Z020 FPGA platform. The reported results indicated a reduction in parameter count and an improvement in area-delay product compared with LeNet. This work reported reduced unnecessary computations and hardware complexity, motivating by the integration of addition and pooling operations into a single processing unit.

Véstias et al., (2019) proposed an optimized CNN accelerator for resource-constrained FPGAs using fixed-point arithmetic, zero-skipping, dynamic activation pruning, and weight pruning. Their approach reduces computational requirements and memory usage to improve CNN inference performance on embedded FPGA platforms. The proposed architecture was evaluated on Zynq FPGA devices and demonstrated improved execution performance with limited accuracy degradation. The study shows that avoiding unnecessary operations and reducing the amount of data processed can improve hardware efficiency. This zero-skipping technique primarily targets zero-valued activations and their pruning technique reduces redundant weights.

Jiang et al. (2021) proposed an FPGA-based CNN optimization framework that combines convolution and pooling operations. Their approach eliminates redundant

multiplications and reuses intermediate addition results. By optimizing multiple processing stages, the framework reduces arithmetic operations and data communication. The authors reported up to 75% reduction in multiplication and addition operations in the best case. The study demonstrates the benefits of combining related operations to improve hardware efficiency.

2.1 INFERENCES FROM LITERATURE REVIEW

The reviewed papers demonstrate that hardware CNN accelerators are efficient through parallel execution, efficient buffering, quantization, and reduced model complexity. Custom convolution modules and pipelined data processing can improve execution speed, while smaller data representations and optimized arithmetic operations can reduce hardware resource requirements. However, memory access, redundant computations, and hardware utilization remain important design challenges. Based on these findings, the proposed project evaluates three hardware modules—PQP convolution, function-merging add-pool, and SZW to study their functionality and contribution to efficient FPGA implementation.

CHAPTER 3
METHODOLOGY

3.1 Methodology Overview

The project focuses on the hardware implementation of modules of a dual branch 2D Convolutional Neural Network for ECG signal processing. This work incorporates Quantisation techniques in hardware design of convolution modules. Further pruning is achieved using Skip Zero Weight techniques thereby improving the latency of the throughput in ECG based Arrythmia Classification

3.2 Workflow of System Architecture

This project utilises the MIT-BIH Arrhythmia Database to prepare ECG beat samples for classification. The raw ECG recordings are processed through filtering, normalization, R-peak-based beat extraction, and waveform-to-image conversion. The resulting grayscale ECG images are resized to 64 × 64 pixels. The model is trained to distinguish five ECG beat categories: Normal (N), Supraventricular ectopic (S), Ventricular ectopic (V), Fusion (F), and Unknown (Q). The trained model provides a software reference for evaluating the effectiveness of reduced-precision computation and the corresponding hardware-oriented architecture.

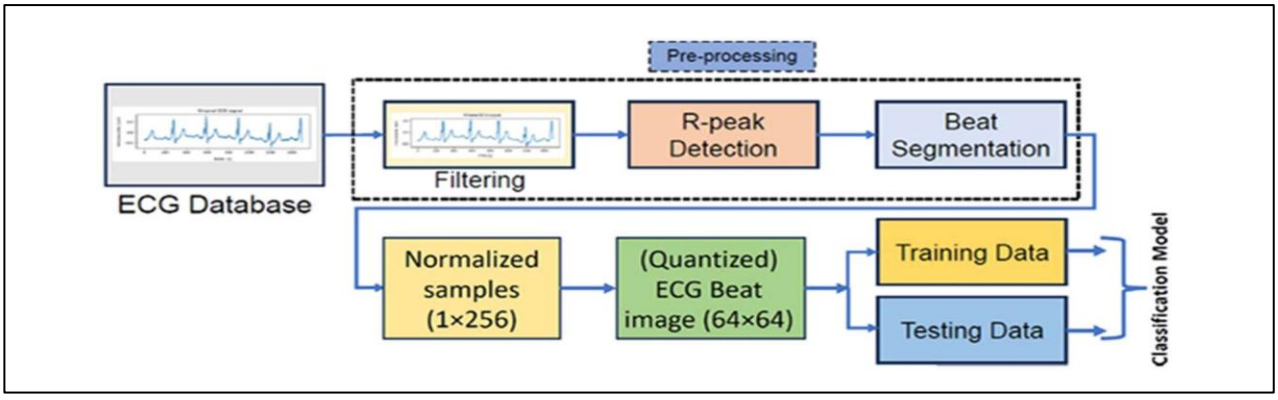


Figure 3.1 Software workflow

The hardware design of convolution modules for the extraction of local features, and extraction of patterns over a different receptive field is attempted. To improve hardware efficiency, the architecture incorporates three principal computational optimizations: PQP convolution, Skip-Zero-Weight (SZW) processing, and a function-merging Add-Pool unit. The PQP module performs pixel-wise multiplication and accumulation in parallel across the relevant convolution weights. The SZW module avoids unnecessary multiply-accumulate operations when the corresponding filter weight is zero. The Add-Pool unit combines the feature maps from the two branches and performs pooling, reducing the need to store intermediate results between these operations. The proposed hardware architecture also uses reduced-precision arithmetic to limit the data width and computational requirements of the FPGA implementation. The Verilog implementation is divided into functional modules for convolution processing, memory access, weight mapping, arithmetic operations, and output generation. These modules can be simulated and verified independently before integration. Further these modules are exported to IP block and incorporated in ZYNQ Ultrascale processing System to transform into full PS-PL block design and finally generate bitstream of hardware hands off file. These files are utilised for hardware accelerated inference using PYNQ Overlay.

The overall development process therefore proceeds from ECG dataset preparation and Software model development to hardware-oriented arithmetic design, RTL simulation, and FPGA implementation. The intermediate outputs generated by the hardware modules can be compared against the corresponding software reference outputs to verify functional correctness.

3.3 Software and Hardware Requirements:

The project requires software tools for ECG dataset processing, CNN model development, RTL coding, and hardware verification. ECG preprocessing, model training, and classification is done in software model training, while Verilog HDL is used to design the hardware modules. Xilinx Vivado is used for RTL simulation, synthesis, and FPGA implementation. The PYNQ-Z2 development board is selected as the target platform for the hardware design. Its Zynq-Ultra scale SoC combines a processing system with programmable logic, enabling software control alongside custom digital hardware implementation.

3.4 ECG Dataset Preparation and Pre - Processing

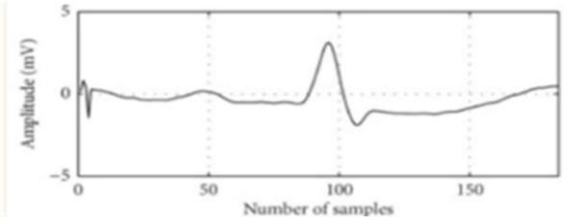
MIT-BIH Arrhythmia database is used to develop the ECG classification model. It contains ECG recordings, header information, and beat annotations that identify important waveform events and their corresponding labels. MIT-BIH Arrhythmia Database contains three main file types: .dat files, which store digitized ECG signal samples recorded from patients; .hea files, which provide header information such as sampling frequency, signal names, and analog-to-digital converter (ADC) details; and .atr files, which contain annotations identifying heartbeat locations and their corresponding classification labels.

Table 3.1 Dataset Details

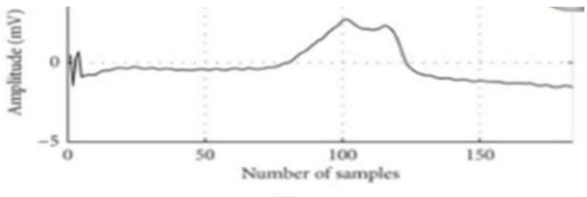
Number of records	48 ECG recordings
Recording duration	30 minutes per record
Sampling frequency	360 Hz
Total Dataset Size	52858 Beats
Maximum signal amplitude	10mV

The preprocessing stage consists of filtering and normalizing the ECG signals, extracting heartbeat segments around annotated R-peaks, and assigning the corresponding classification labels. Each extracted heartbeat is represented as a grayscale image and resized to 64 × 64 pixels for use as input to the Software CNN model. Quantization is subsequently applied to prepare reduced-precision data for hardware-oriented processing.

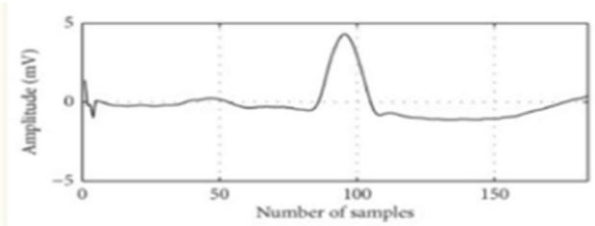
The five classification categories used in the project are N, S, V, F and Q in which N represents a normal heartbeat, S represents a supraventricular ectopic beat, which originates above the ventricles. V represents a ventricular ectopic beat, which originates in the ventricles. F represents a fusion beat, which combines characteristics of normal and ventricular beats. Q represents an unknown, paced, or unclassifiable beat that cannot be clearly assigned to the other categories.



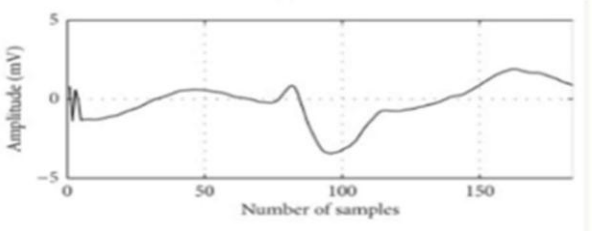
Normal N



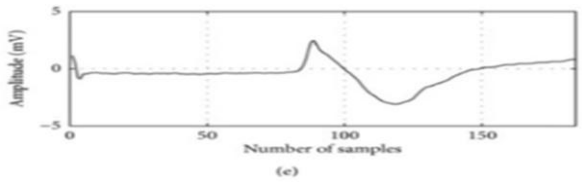
Supraventricular Ectopic S



Fusion F



Ventricular Ectopic V



Unknown Q

Figure 3.2 Representation of ECG signals for Arrythmia classification

3.5 Dual-Branch CNN Architecture Design:

The dual-branch convolution architecture is designed to extract complementary features from ECG images while improving the potential for parallel hardware execution. The input is processed simultaneously by two branches with different convolution paths. Branch 1 performs two successive convolution operations, whereas Branch 2 performs a separate convolution operation using a different kernel size. The outputs of the branches are combined through an addition operation, followed by max pooling to reduce the spatial dimensions of the resulting feature maps. This arrangement allows the system to combine local features from successive convolutions with features extracted through the alternate branch.

The PQP convolution method is incorporated to improve data reuse and reduce repeated pixel fetching. Instead of processing the relevant feature-map pixels independently for every filter, the architecture enables shared pixel data to participate in parallel multiplication operations across multiple kernels. This design forms the basis for the Verilog convolution modules developed in the project.

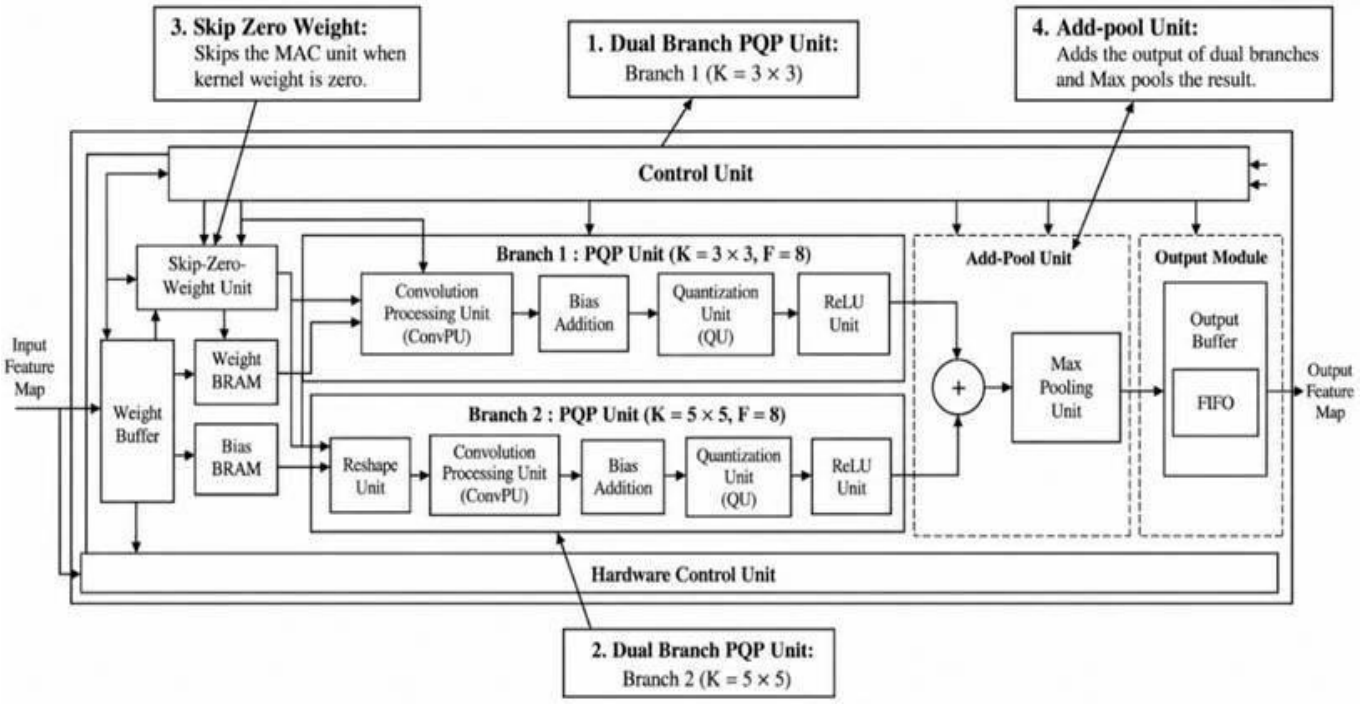


Figure 3.3 Hardware Control Architecture

The dual-branch CNN architecture depicted in figure 3.3 processes the input ECG beat image through two parallel convolution paths to extract relevant features. The input image is applied to both branches. In the first branch, two successive convolution operations with 3x3 kernels are performed, while the second branch uses a 5x5 convolution kernel to capture features over a larger receptive field. The Skip-Zero-Weight (SZW) mechanism is incorporated into the convolution operations to avoid unnecessary multiply-accumulate computations when the corresponding kernel weight is zero. The outputs of both branches are combined using an adder and passed to the max-pooling stage. These operations are integrated into the Add Pool unit, which combines feature-map addition and spatial downsampling.

Table 3.2 Hyperparameters of each module

Layer Type	Input Dimension	Output Dimension	Connected to
Input Layer (Image)	512 x 512	_	_
Conv_l1 (5 x 5)	512 x 512	508 x 508, 8	Input Layer
Conv_l2 (3 x 3)	512 x 512	510 x 510, 8	Input layer
Conv_l3 (3 x 3)	510 x 510	508 x 508, 8	Conv_l2
Adder	508 x 508, 8	508 x 508, 8	Conv_l3, Conv_l1
Max Pool	508 x 508, 8	254 x 254, 8	Adder

The Table 3.3 presents the layer-wise parameters involved in the proposed dual-branch 2D-CNN. The input image of size 512x512 is processed through two convolution branches. Branch 1 consists of two successive 3x3 convolution layers (Conv_l2 and Conv_l3), generating eight feature maps of size 508x508. Branch 2 uses a 5x5 convolution (Conv_l1), producing eight feature maps of the same spatial dimensions. The outputs of both branches are combined using an adder to integrate the extracted features. Finally, a max-pooling operation reduces the feature-map dimensions to 254x254, decreasing spatial data while retaining important features for subsequent processing.

3.5.1 Skip Zero Weight (SZW) Unit Architecture:

The Skip-Zero-Weight (SZW) architecture is designed to reduce unnecessary arithmetic operations during convolution by identifying kernel weights with a value of zero. In conventional convolution, each input pixel is multiplied by its corresponding

kernel weight, and the products are accumulated to generate the output. However, when a kernel weight is zero, the multiplication contributes nothing to the accumulated result.

The SZW unit checks the kernel weights before the multiplication and accumulation process. If a weight is zero, the corresponding multiply-accumulate (MAC) operation can be skipped. If the weight is non-zero, the input pixel is multiplied by the weight, and the resulting product is accumulated. This approach can reduce unnecessary switching activity and computational effort in the hardware datapath.

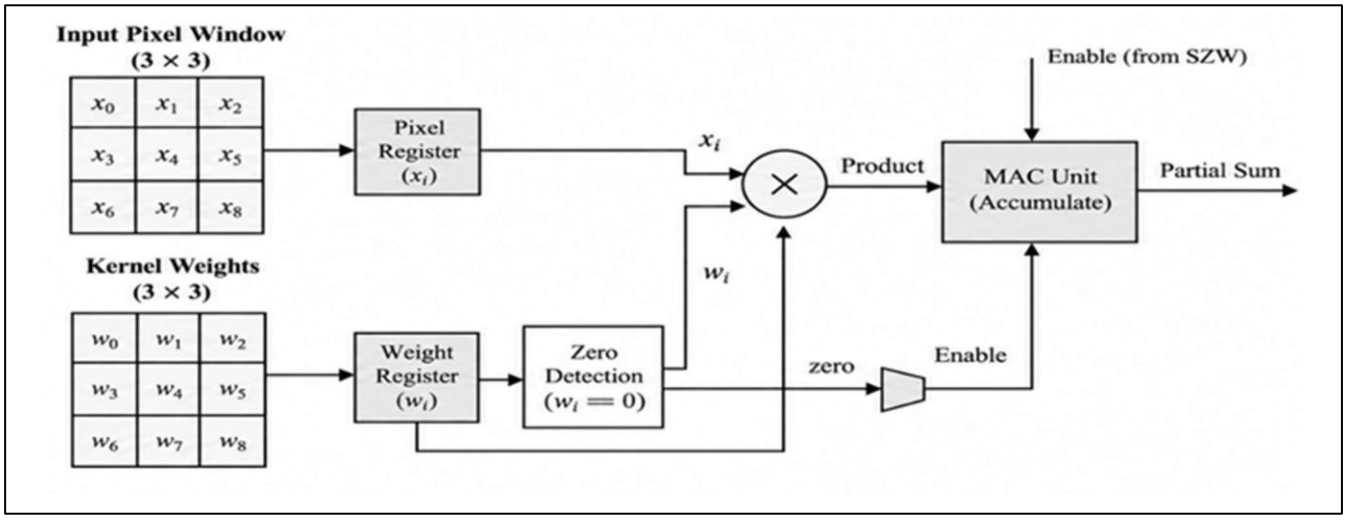


Figure 3.4 Skip Zero Weight Architecture

In the proposed PQP convolution architecture, SZW processing complements parallel pixel-wise computation by avoiding redundant operations associated with zero-valued weights. Its effectiveness depends on the number of zero weights and how the skip control is implemented in the RTL design.

3.5.2 Parallel-Quantized-Pixel-wise (PQP) convolution architecture:

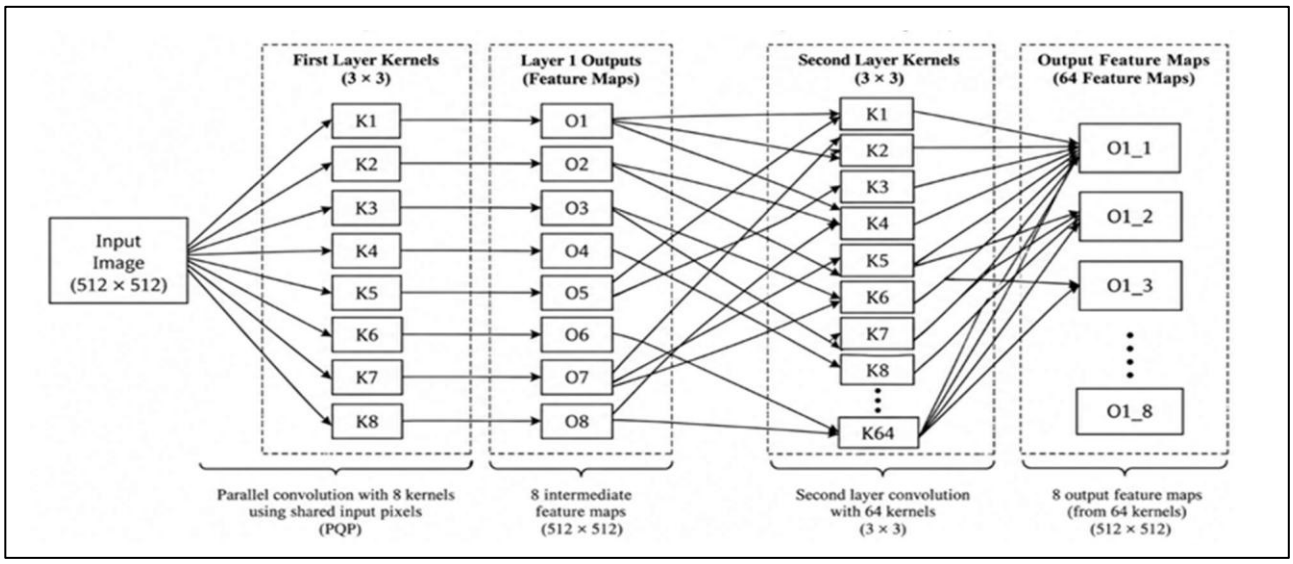


Figure 3.5 Architectural Flow of PQP Convolution Layers

Parallel-Quantized-Pixel-wise (PQP) convolution architecture depicted in Figure 3.5 is designed to perform convolution operations efficiently by reusing input pixel values across multiple kernel computations. As illustrated in the figure, the input image is processed by eight first-layer kernels of size 3x3, generating eight intermediate feature maps. These feature maps are then supplied to the second convolution layer, which uses 64 kernels to perform further feature extraction.

The PQP approach enables parallel processing of shared input pixels, reducing repeated pixel fetching and providing opportunities for improved hardware utilization. The convolution modules are designed to support implementation in Verilog HDL, where the pixel values and kernel weights are processed through arithmetic units to generate the corresponding output feature maps.

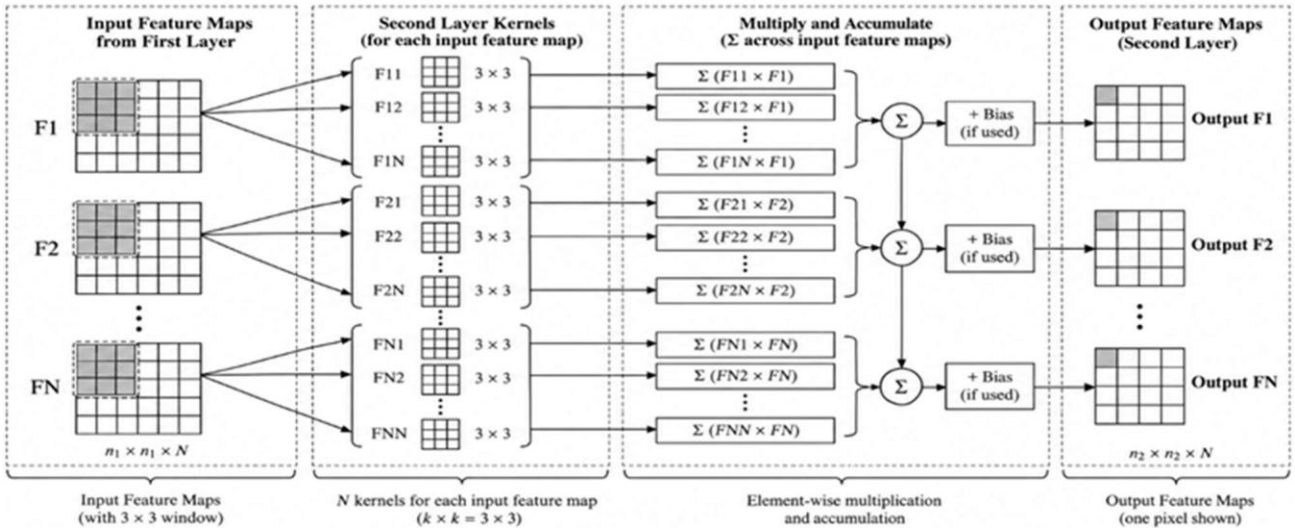


Figure 3.6 PQP Layer II Architectural Flow

The second layer of the Parallel-Quantized-Pixel-wise (PQP) convolution architecture receives the feature maps generated by the first convolution layer. Each input feature map is processed using multiple 3x3 kernels, allowing convolution operations to be performed across the input channels.

As shown in the Figure 3.6, the input feature maps are connected to their corresponding second-layer kernels. Each kernel performs element-wise multiplication with the relevant input region, and the resulting products are accumulated to generate a partial convolution result. The partial results from the input feature maps are then summed to obtain the output feature map. A bias term is added if required by the implemented design. The PQP approach supports parallel kernel computation and efficient reuse of input pixel data. This reduces repeated data access and provides opportunities for improved hardware efficiency. The resulting feature maps are passed to the next processing stage of the CNN.

3.5.3 Add-Pool Unit Architecture:

The Add-Pool unit combines the feature maps generated by the two branches of the dual-branch PQP convolution architecture. It performs element-wise addition followed by max pooling, allowing the features extracted by both branches to be merged into a single set of feature maps. The outputs from Branch 1, which uses 3x3 convolution kernels, and Branch 2, which uses 5x5 convolution kernels, are supplied to the addition unit. For element-wise addition, both branches must produce feature maps with compatible spatial dimensions and the same number of channels. The corresponding values are added to generate the merged feature maps.

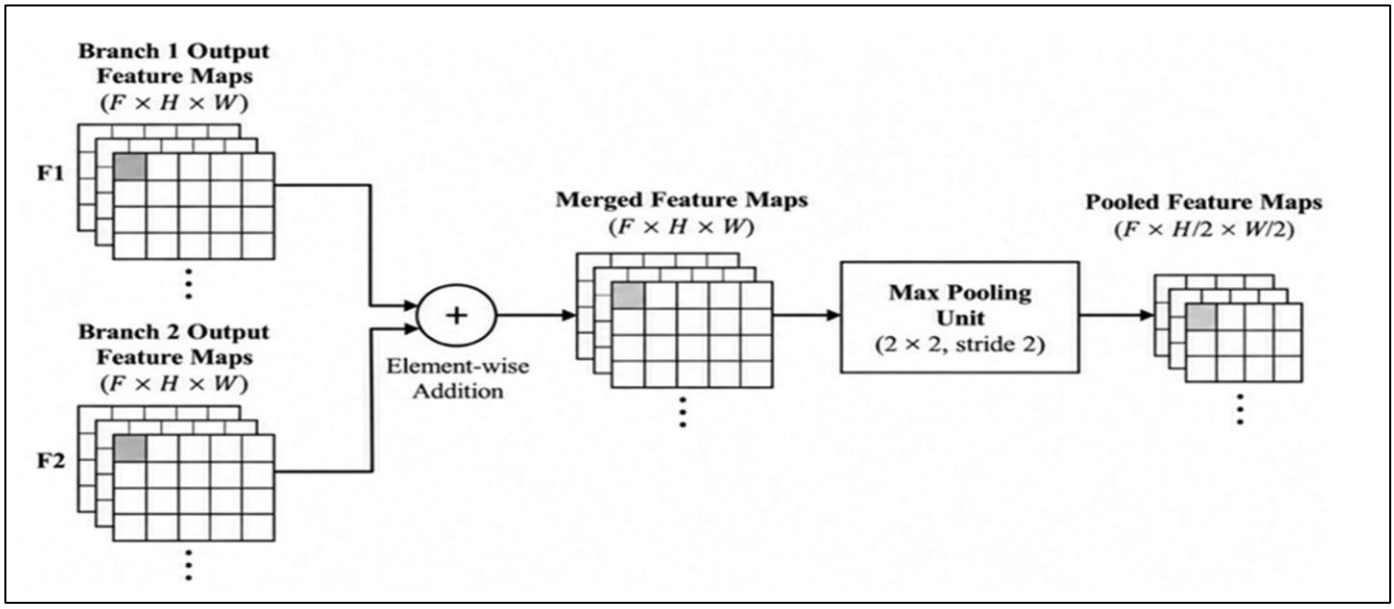


Figure 3.7 Add-Pool Architecture

The merged feature maps are then passed through a 2x2 max-pooling operation with a stride of 2. The maximum value within each pooling window is selected, reducing the height and width of the feature maps by approximately half while retaining prominent feature responses. The pooled outputs are then forwarded to the next processing stage or

output buffer. By combining addition and pooling into a single functional unit, the architecture can reduce intermediate data handling and simplify the overall processing flow.

3.6 Dual-Branch CNN Model Development:

The dual-branch CNN model is developed using Python programming language to classify ECG beat images into five ECG beat categories. The model processes the input through two parallel branches to extract complementary features. One branch performs sequential convolution operations to identify local waveform patterns, while the other applies a separate convolution operation to capture features over a larger receptive field.

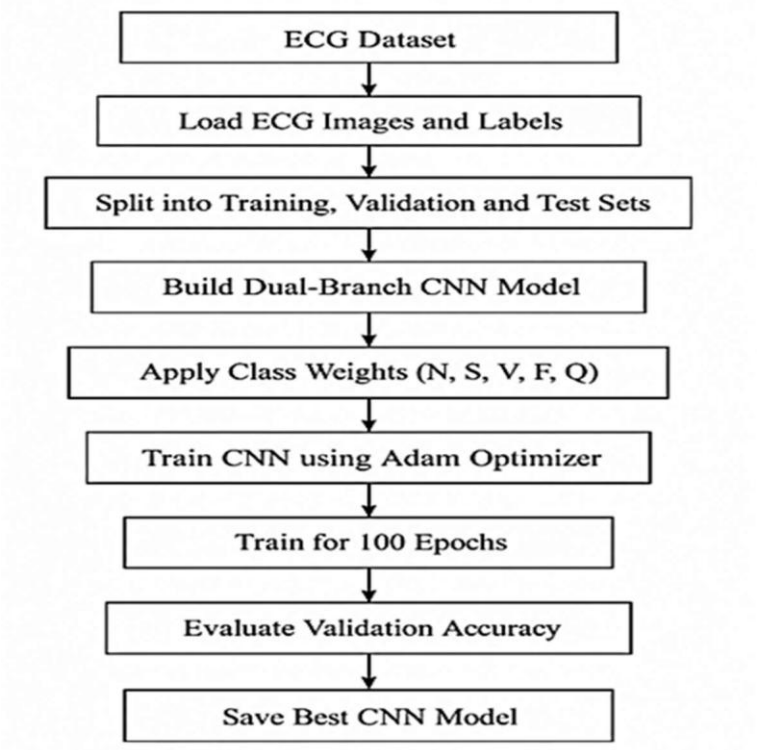


Figure 3.8 Workflow of CNN training

The model is trained using the prepared ECG image dataset. Training involves dividing the dataset into training, validation, and testing subsets, applying class weights to address class imbalance, and optimizing the network parameters using the Adam

optimizer. The trained model is evaluated using the test dataset, and its weights and biases provide reference parameters for hardware-oriented implementation.

3.7 PS - PL IP Design and Hardware Deployment:

The PYNQ-Z2 development board is selected as the target platform for implementing the proposed Dual-Branch Parallel-Quantized-Pixel-wise (PQP) convolution architecture. The board is based on the Zynq-7000 System-on-Chip (SoC), which integrates a Processing System (PS) containing an ARM Cortex-A9 processor and Programmable Logic (PL) for implementing custom digital hardware. FPGAs are particularly suitable for CNN acceleration because their configurable logic allows multiple arithmetic operations to be performed in parallel, offering opportunities to reduce processing latency and improve computational efficiency compared with purely sequential execution.

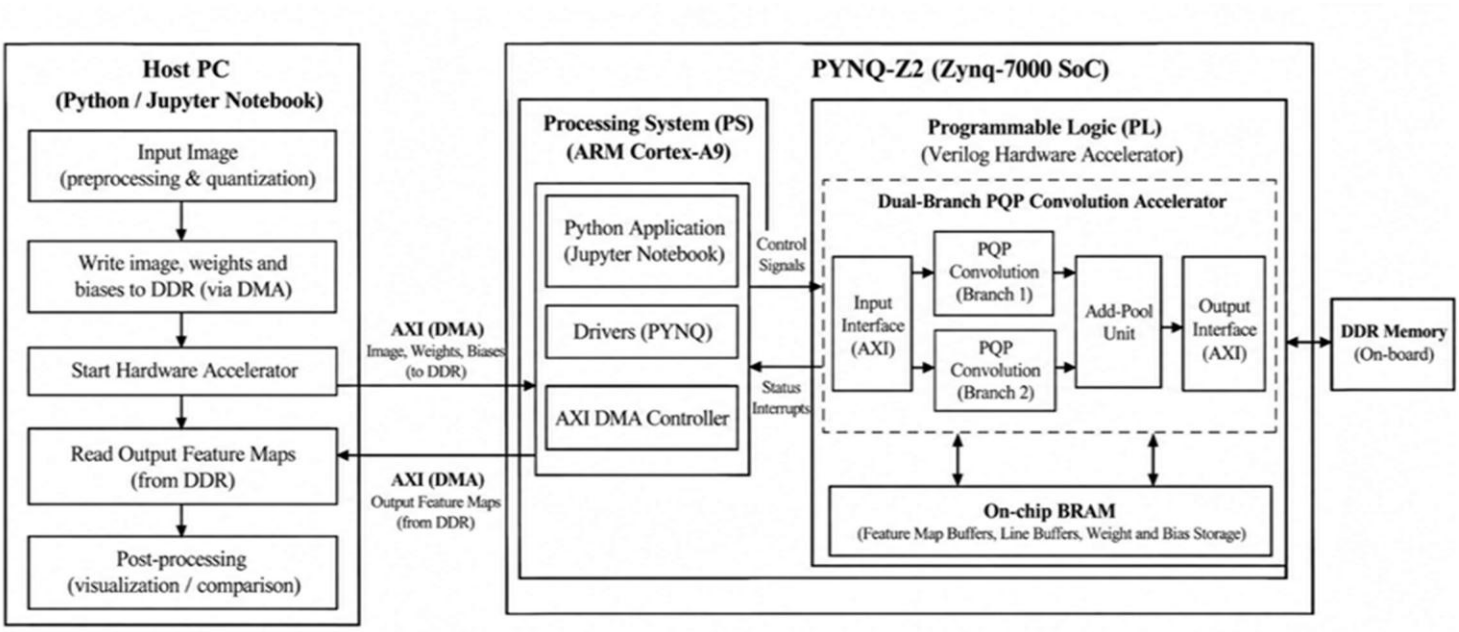


Fig 3.9 PS – PL Integrated architecture design

As illustrated in the Figure 3.8, the host PC provides the software environment for input preparation, quantization, and result analysis using Python Overlays in Jupyter

Notebook. The ECG image is pre - processed and converted into the required numerical representation before being transferred, along with the convolution weights and biases, to the board's memory. The software initiates the hardware accelerator and manages data transfers through the AXI Direct Memory Access (DMA) interface. DMA enables image data and model parameters to be transferred between memory and the hardware processing blocks with reduced processor involvement in the data-transfer process. Once processing is complete, the output feature maps are retrieved from memory and passed back to the host PC for visualization and comparison with the software reference outputs.

Within the Programmable Logic, the Dual-Branch PQP Convolution Accelerator forms the main computational unit. The input interface receives the image data and supplies it to two parallel convolution branches. PQP architecture is designed to improve parallelism by allowing multiple pixel–weight multiplication operations to be performed concurrently and by reducing repeated fetching of input pixel data. The two branches capture complementary feature information, which is subsequently combined by the Add-Pool unit. This unit performs feature-map addition followed by max pooling, integrating the branch outputs and reducing their spatial dimensions. Combining these operations within a single functional block is intended to minimize intermediate data movement and reduce the need for separate processing stages. The Skip-Zero-Weight (SZW) mechanism complements the PQP convolution architecture by identifying zero-valued filter weights and avoiding the corresponding unnecessary multiply-accumulate (MAC) operations. On-chip block RAM (BRAM) provides storage for intermediate feature maps, line buffers, weights, and biases, allowing the accelerator to access frequently used data locally. The output interface transfers the processed feature maps to the memory system for subsequent retrieval and analysis. The PS coordinates execution and data movement, the AXI interfaces facilitate communication, and the BRAM supports local data storage.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Illustration of the Convolution Operation

The input image shown in Figure 4.1 contains the same pixel value, 15, at all nine positions. The Sobel Y kernel has coefficients whose sum is zero, so the positive and negative weighted contributions cancel, producing an output of zero. This is demonstrated by the sample calculation shown in the figure:

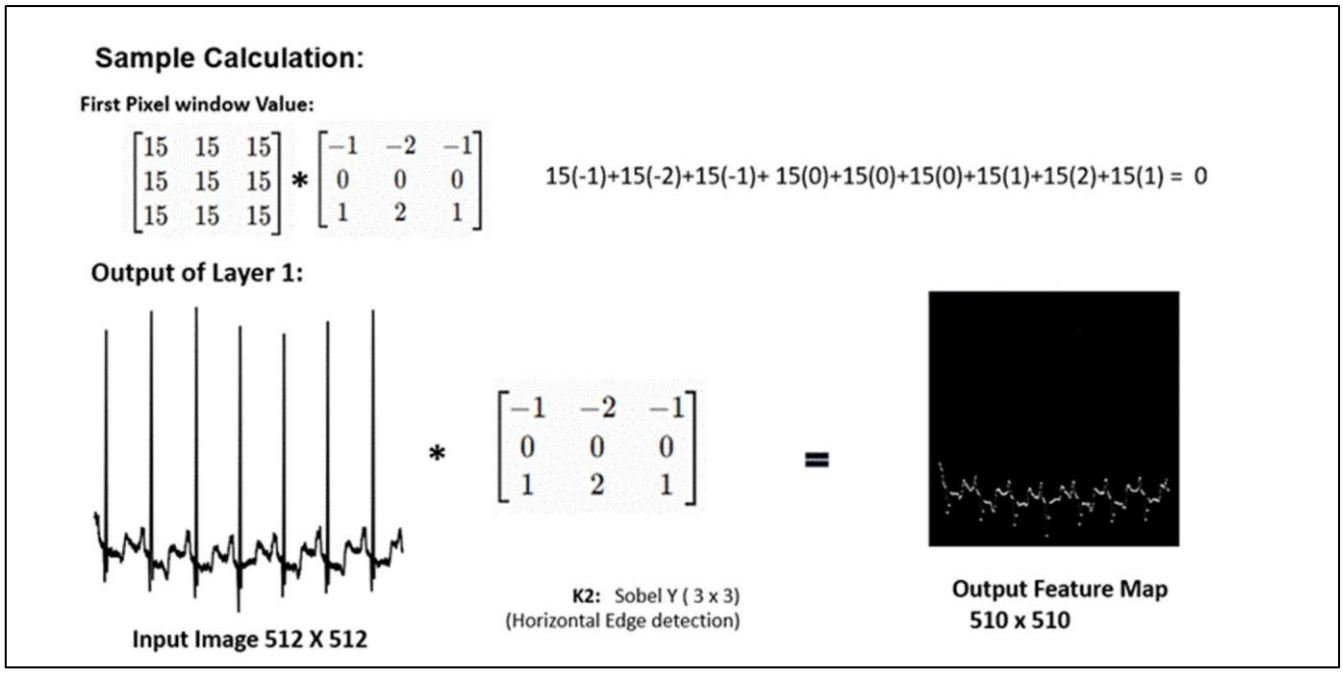


Figure 4.1: Illustration of Convolution operation for ECG input image

As shown in Figure 4.1, the 512x512 ECG input image is convolved with the 3x3 Sobel Y kernel to produce a 510x510 output feature map. The dimensions decrease because valid convolution does not use padding. Although the first window produces zero,

other windows contain varying pixel values and generate non-zero responses, making the ECG waveform edges visible in the output feature map. Thus, the figure 4.2 demonstrates the expected directional edge-detection behaviour of the convolution operation.

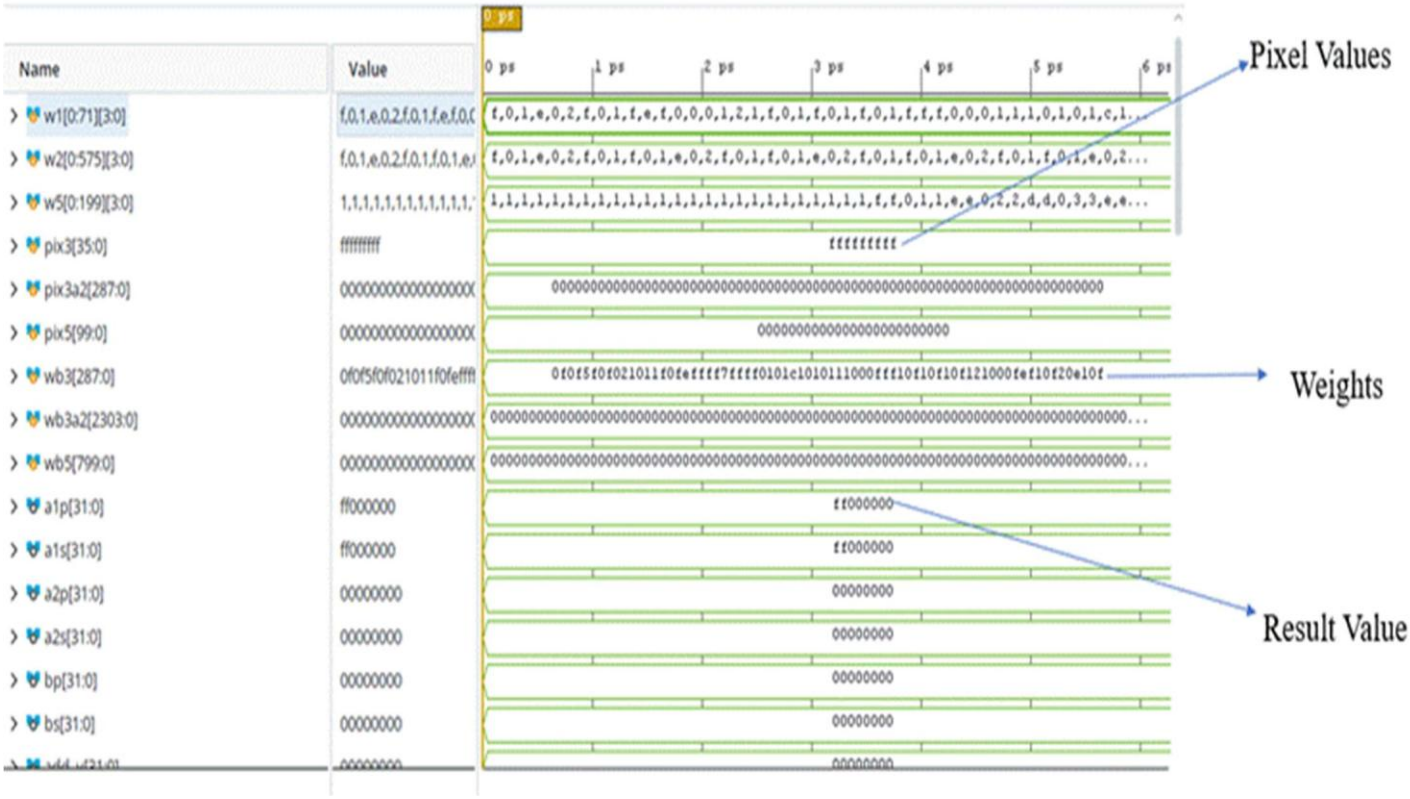


Figure 4.2: RTL Simulation waveform of final convolution layer

The waveform shown in Figure 4.2 illustrates the signals involved in the convolution module during RTL simulation. The signals labelled Pixel Values represent the input pixel data, while the Weights signals represent the kernel coefficients used in the convolution operation. The Result Value signals show the computed outputs at different stages of the processing unit. The waveform displays changes in the input data and corresponding signal values over the simulation timeline, allowing the internal operation of the module to be examined. The result signals shown as hexadecimal values represent the current digital outputs of the hardware. By comparing these outputs with the expected

convolution calculations, the functional behavior of the module can be verified for the tested inputs.

4.2 Dual-Branch Convolution and Feature-Map Addition

The dual-branch architecture generates two feature maps from separate convolution paths, as shown in Figure 4.3.

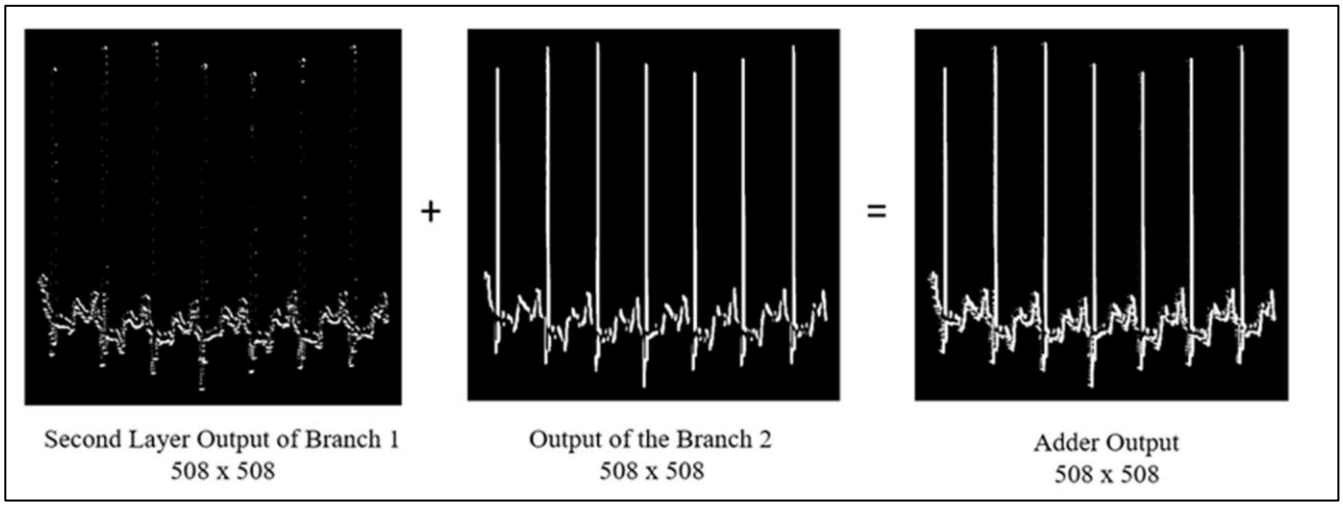


Figure 4.3: ECG image representation of individual branch outputs

The second-layer output of Braanch 1 contains relatively faint waveform details, whereas the output of Branch 2 shows more prominent ECG peaks and waveform variations. These outputs are added element by element to produce the adder output, which retains the prominent peaks while incorporating contributions from both branches.

The feature-map addition can be expressed as

$A(i,j)=B_1(i,j)+B_2(i,j)$

where B_1and B_2 represent corresponding branch outputs and A is the combined feature map.

4.3 Implementation of the Max Pooling Unit

As shown in Figure 4.4, the input feature map of size 508x508 contains prominent ECG peaks and waveform variations. After max-pooling, the output size is reduced to 254x254 while the major waveform structures remain visible. For each non-overlapping 2x2 window, the maximum of the four input values is selected, reducing the spatial dimensions by half along both axes.

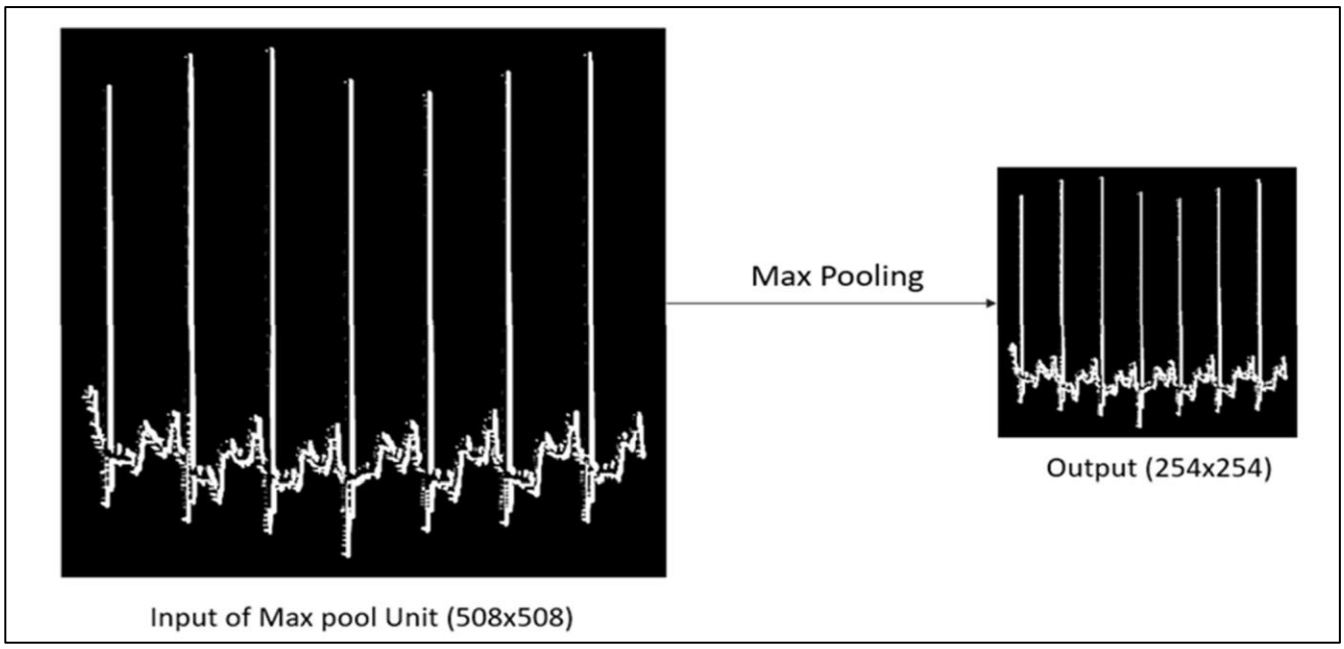


Figure 4.4: Representation of ECG signal before and after Max Pooling Unit

Consequently, the number of feature-map pixels decreases from 258,064 to 64,516, representing a 75% reduction in spatial data. This result demonstrates the spatial down sampling performed by the pooling stage. Combining addition and pooling into a single unit is intended to reduce intermediate data handling between the operations.

4.4 Simulation Waveform of Final Layer

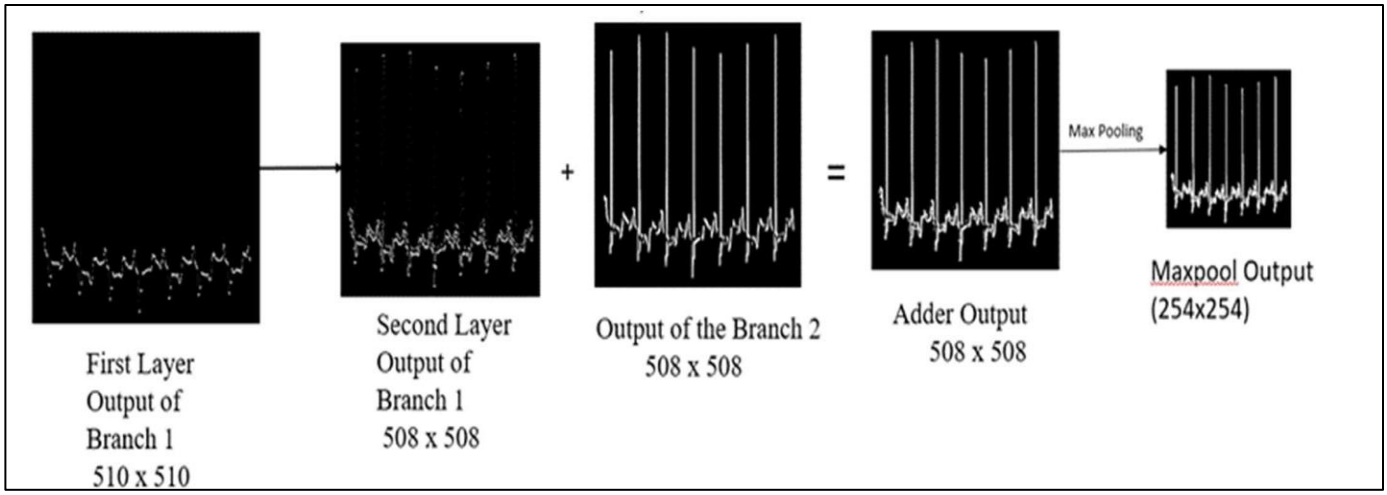


Figure 4.5: Outputs of each layer

As shown in Figure 4.5, the first convolution stage of Branch 1 produces a 510x510 feature map, followed by a second convolution stage that generates a 508x508 output. The output of Branch 2 is also 508x508; both branch outputs are added elementwise, and the resulting feature map is passed through max-pooling to obtain a 254x254 output. The prominent ECG peaks remain visible throughout these stages, indicating that the main waveform structures are retained after convolution, addition, and down sampling.

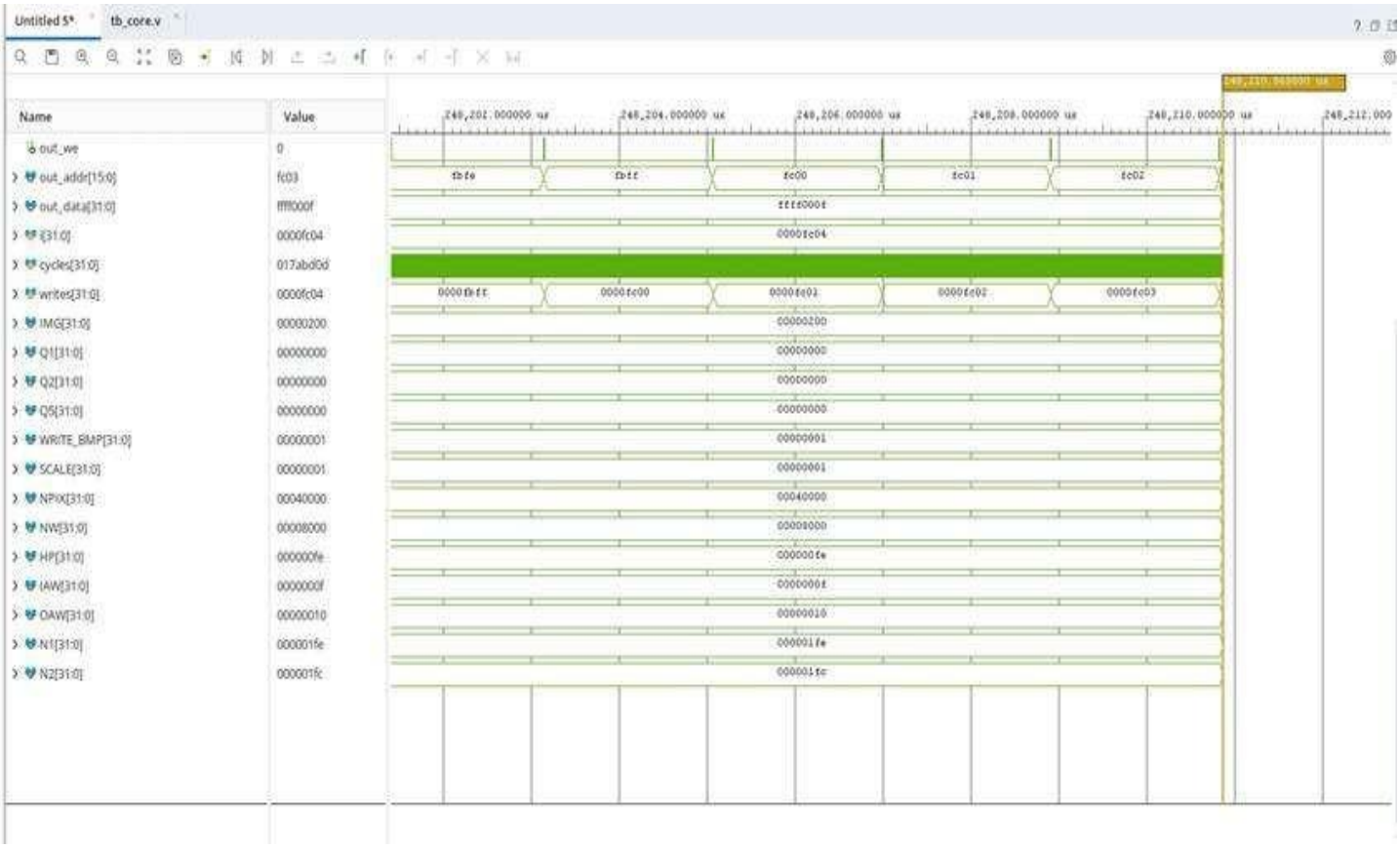


Figure 4.6: Simulation Waveform of Final Layer of PQP Convolution

The Figure 4.6 shows the RTL simulation waveform, where the output address, output data, cycle count and write-control signals are visible during memory operations. The output address progresses and the data is updated, indicating successive output writes during the observed simulation interval. Together, these figures illustrate the layer-wise processing flow and the associated hardware output activity of the dual-branch CNN.

4.5 FPGA Resource Utilization and Evaluation

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	Block RAM Tile (140)	DSPs (220)	Bonded IOB (125)	BUFGCTRL (32)
pqp_axi_top	2616	4596	377	175	2262	2616	100.5	30	114	1
u_core (dual_branch_core)	2417	4343	377	175	2125	2417	4.5	30	0	0
u_e1 (pqp_engine)	225	70	0	0	92	225	0	10	0	0
u_e2 (pqp_engine__paran	232	72	0	0	99	232	0	10	0	0
u_e5 (pqp_engine__paran	292	68	0	0	125	292	0	10	0	0
u_fap (fused_add_pool)	102	168	0	0	81	102	0.5	0	0	0
rowbuf (linebuf__paran	83	2	0	0	31	83	0.5	0	0	0
u_l1 (szw_list)	162	299	0	0	148	162	0	0	0	0
u_l2 (szw_list__parameter	858	2440	271	132	1137	858	0.5	0	0	0
u_l5 (szw_list__parameter	277	730	54	27	381	277	0.5	0	0	0
u_wg2 (window_gen__par	25	272	0	0	76	25	1	0	0	0
u_wg5 (window_gen)	58	106	16	0	42	58	2	0	0	0

Figure 4.7: Resource utilization of PQP Convolution unit

The utilization report for the pqp_axi_top design shown in figure 4.7 indicates 2,616 LUTs, 4,596 registers, 30 DSPs and 100.5 block-RAM tiles. The LUT and register usage reflects the additional hardware required to support parallel arithmetic, data storage and control operations in the PQP architecture. The 30 DSPs indicate the use of dedicated arithmetic resources for computation, while the block RAM stores intermediate or input data.

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	Block RAM Tile (140)	DSPs (220)	Bonded IOB (125)	BUFGCTRL (32)
seq_axi_top	1885	4501	516	236	1351	1885	99.5	16	114	1
u_core (dual_branch_seq_core)	1673	4247	516	236	1194	1673	3.5	16	0	0
u_e1 (conv_seq_engine)	130	53	36	16	48	130	0	5	0	0
u_e2 (conv_seq_engine__parar	683	56	304	144	210	683	0	6	0	0
u_e5 (conv_seq_engine__parar	347	54	104	52	126	347	0	5	0	0
u_fap (fused_add_pool)	101	168	0	0	82	101	0.5	0	0	0
rowbuf (linebuf__parameter	83	2	0	0	31	83	0.5	0	0	0
u_w1 (wreg_loader)	31	296	0	0	89	31	0	0	0	0
u_w2 (wreg_loader__paramete	85	2315	16	8	589	85	0	0	0	0
u_w5 (wreg_loader__paramete	29	809	0	0	200	29	0	0	0	0
u_wg2 (window_gen__paramet	24	272	0	0	71	24	1	0	0	0
u_wg5 (window_gen)	53	106	20	0	32	53	2	0	0	0

Figure 4.8: Resource utilisation of Standard Convolution

The utilization report for the seq_top design shown in Figure 4.8 indicates 1,885 LUTs, 4,501 registers, 16 DSPs and 99.5 block-RAM tiles. Compared with the PQP design, seq_top uses 731 fewer LUTs, 95 fewer registers and 14 fewer DSPs. This lower LUT and DSP usage is consistent with sequential processing, which reuses hardware resources across operations rather than performing multiple operations simultaneously. The PQP architecture requires additional parallel datapaths and associated control logic, increasing its hardware resource requirements in exchange for greater processing parallelism.

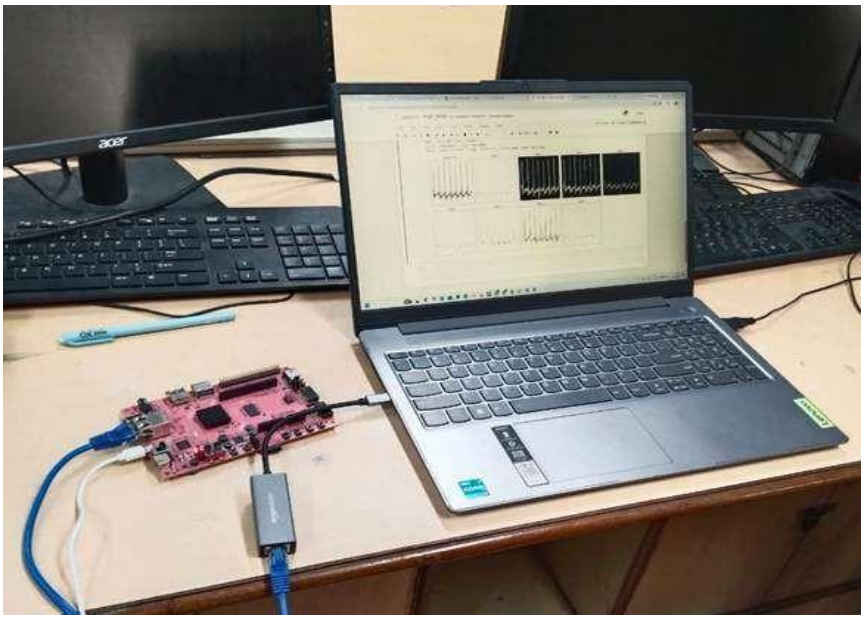


Figure 4.9 : Experimental Setup

The experimental setup shown in Figure 4.9 illustrates the FPGA development board connected to the host computer for the hardware demonstration. Table 4.1 summarises the comparison between the PQP + SZW and sequential convolution implementations in terms of FPGA resource utilization and processing cycles. The PQP + SZW design uses 30 DSPs (13.6%) and 2,616 LUTs (4.91%), while the sequential design uses 16 DSPs (7.27%) and 1,885 LUTs (3.54%). Their register usage is comparable, at 4,596 and 4,501, respectively.

Table 4.1: Comparison of PQP and Standard convolution

	PQP + SZW	Sequential
DSP (220)	30 (13.6%)	16 (7.27%)
LUTs (53200)	2616 (4.91%)	1885 (3.54%)
Registers (106400)	4596 (4.32%)	4501 (4.23%)
Cycles Per frame	24.8M	170.8M
Time at 100MHz	248 ms	1707 ms

From the Table 4.1 we can infer that PQP + SZW requires 24.8 million cycles per frame compared with 170.8 million for sequential processing, corresponding to reported processing times of 248 ms and 1,707 ms at 100 MHz. These results indicate that PQP + SZW achieves a substantially lower cycle count at the cost of higher LUT and DSP utilization.

4.6 Software Model Training:

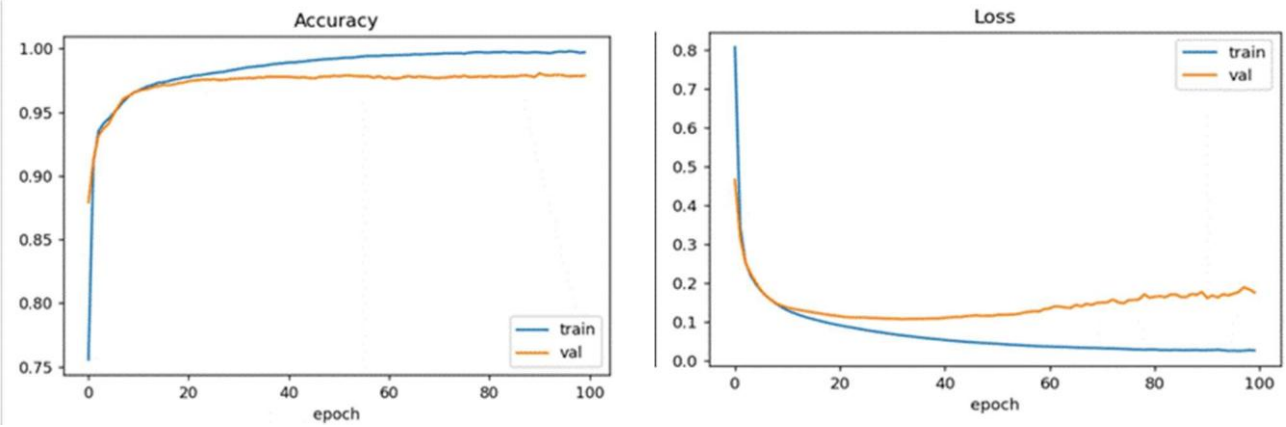


Fig 4.10: Performance curve of trained model

As shown in Figure 4.10, the software-trained ECG classification model was trained

for 100 epochs to classify heartbeats into five categories: N, S, V, F and Q. The accuracy curve shows that training and validation accuracy increase rapidly during the initial epochs and then stabilize, with training accuracy approaching 99%. The loss curve shows a continuous decrease in training loss, while validation loss decreases initially and rises slightly during later epochs, suggesting possible overfitting. The model achieved a best validation accuracy of 98.06% and a test accuracy of 97.74%, demonstrating strong classification performance before hardware quantization.

4.6.1 Classification Performance

The confusion matrix and classification report shown in Figure 4.11 evaluate the model's performance across the five ECG heartbeat classes (N, S, V, F and Q). The confusion matrix shows that most samples are correctly classified along the diagonal, while the off-diagonal values represent misclassifications. The model achieved a test accuracy of 97.74% on 7,929 heartbeats, with precision, recall and F1-scores providing additional insight into class-wise performance. The results indicate strong overall classification performance, although the S and F classes show comparatively lower recall or F1-scores.

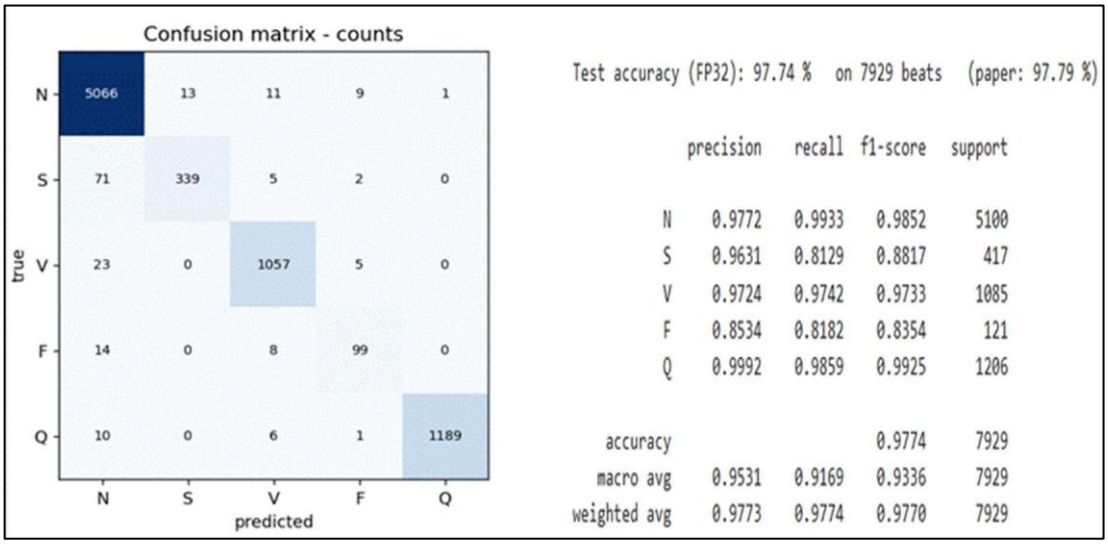


Fig 4.11: Confusion Matrix and Class-Wise Performance Metrics of the ECG Classification Model.

4.6.2 Classification Performance after INT4 Quantization:

```
EXPORT: weights asym 4-bit, activations 4-bit
Test accuracy   float model file: 97.74 %    integer model: 92.65 %
              precision    recall  f1-score   support

           N     0.9741    0.9502    0.9620      5100
           S     0.8990    0.4484    0.5984       417
           V     0.8040    0.9714    0.8798      1085
           F     0.4924    0.5372    0.5138       121
           Q     0.9163    0.9900    0.9518      1206

    accuracy                         0.9265      7929
   macro avg     0.8172    0.7795    0.7812      7929
weighted avg     0.9307    0.9265    0.9232      7929

confusion matrix (rows true, cols predicted):
 [[4846   19   86   57   92]
 [  96  187  122    5    7]
 [  16    1 1054    5    9]
 [  13    1   41   65    1]
 [   4    0    8    0 1194]]
```

Fig 4.12: Classification Performance after quantisation

The quantized model's classification results shown in Figure 4.12 indicate a test accuracy of 92.65% using 4-bit weights and activations, compared with 97.74% for the original FP32 model. The confusion matrix and class-wise metrics show how reduced precision affects classification across the five heartbeat categories.

Table 4.2: Summary of Performance with different activation precision

Activation Precision	**Type**	**Test Accuracy**
32 bits	Floating point	97.74%
6 bits	Integer	95.50%
4 bits	Integer	92.65%

Table 4.2 summarizes the test accuracy at different activation precisions, showing 97.74% for FP32, 95.50% for 6-bit integer, and 92.65% for 4-bit integer. This demonstrates the trade-off between numerical precision and classification accuracy when preparing the model for resource-constrained hardware implementation.

CHAPTER 5

CONCLUSION

The project presents a hardware-efficient Dual-Branch 2D-CNN architecture implemented on the PYNQ-Z2 FPGA board. It incorporates three key hardware modules: Parallel-Quantized-Pixel-wise (PQP) convolution, function-merging add-pool, and Skip-Zero-Weight (SZW). The PQP module enables parallel convolution processing, the add-pool unit combines addition and pooling operations to reduce intermediate memory access, and the SZW architecture skips unnecessary computations for zero-valued filter weights.For a (512 X 512) input frame, the PQP + SZW design required 24.8 million simulated cycles, compared with 170.8 million cycles for sequential re-fetch processing. At 100 MHz, these computation corresponds to approximately 248 ms and 1,708 ms, respectively, indicating an 85.5% reduction in cycle count and a 6.9-fold difference in cycle count. The trained ECG classification model is also quantized to INT4 precision, achieving 92.65% test accuracy compared with 97.74% for the original FP32 model, a decrease of 5.09 percentage points. These results illustrate the trade-off between numerical precision and classification accuracy. Overall, the project evaluates the functionality of the proposed hardware modules, their processing performance, and the benefits of parallel computation and reduced-precision operations for FPGA implementation.

CHAPTER 6

REFERENCES

1. Aarrestad, T. et al. (2021) ‘Fast Convolutional Neural Networks on FPGAs with hls4ml’, *Machine Learning: Science and Technology*, Vol. 2, No. 4, Art. No. 045015.
2. Blott, M., Preußer, T.B., Fraser, N.J., Gambardella, G., O’Brien, K., Umuroglu, Y., Leeser, M. and Vissers, K. (2018) ‘FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks’, *ACM Transactions on Reconfigurable Technology and Systems*, Vol. 11, No. 3.
3. Boutros, A. and Betz, V. (2021) ‘FPGA Architecture: Principles and Progression’, *IEEE Circuits and Systems Magazine*, Vol. 21, No. 2, pp. 4–29.
4. Cong, J., Liu, B., Neuendorffer, S., Noguera, J., Vissers, K. and Zhang, Z. (2011) ‘High-Level Synthesis for FPGAs: From Prototyping to Deployment’, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 30, No. 4, pp. 473–491.
5. Degirmenci, M., Ozdemir, M., Izci, E. and Akan, A. (2022) ‘Arrhythmic Heartbeat Classification Using 2D Convolutional Neural Networks’, *IRBM*, Vol. 43, No. 5, pp. 422–433.
6. Erdem, A., Silvano, C., Boesch, T., Ornstein, A., Singh, S.-P. and Desoli, G. (2018) ‘Design Space Exploration for Orlando Ultra Low-Power Convolutional Neural Network SoC’, in *Proceedings of the IEEE 29th International Conference on Application-Specific Systems, Architectures and Processors (ASAP)*, pp. 1–7.
7. Hailesellasie, M., Hasan, S.R., Khalid, F., Wad, F.A. and Shafique, M. (2018) ‘FPGA-Based Convolutional Neural Network Architecture with Reduced Parameter Requirements’, in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*.
8. Huynh, T.V. (2022) ‘FPGA-Based Acceleration for Convolutional Neural Networks

on PYNQ-Z2', *International Journal of Computing and Digital Systems*, Vol. 11, No. 1, pp. 441–449.

9. Jiang, B., Cheng, X., Tang, S., Ma, X., Gu, Z., Zhao, H. and Fu, S. (2021) 'APCNN: Explore Multi-Layer Cooperation for CNN Optimization and Acceleration on FPGA', in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pp. 146–147.
10. Liao, Y., Adegbija, T. and Lysecky, R. (2022) 'A High-Level Synthesis Approach for Precisely Timed, Energy-Efficient Embedded Systems', *Sustainable Computing: Informatics and Systems*, Vol. 35, Art. No. 100741.
11. Li, S., Sun, K., Luo, Y., Yadav, N. and Choi, K. (2020) 'Novel CNN-Based AP2D-Net Accelerator: An Area and Power Efficient Solution for Real-Time Applications on Mobile FPGA', *Electronics*, Vol. 9, No. 5, Art. No. 832.
12. Lu, J., Liu, D., Cheng, X., Wei, L., Hu, A. and Zou, X. (2022) 'An Efficient Unstructured Sparse Convolutional Neural Network Accelerator for Wearable ECG Classification Device', *IEEE Transactions on Circuits and Systems I: Regular Papers*, Vol. 69, No. 11, pp. 4572–4582.
13. Mangaraj, S., Mahapatra, K. and Ari, S. (2025) 'LS-Compiled PYNQ-Based Cardiac Arrhythmia Detection System Leveraging Quantized ECG Beat Images', *Biomedical Signal Processing and Control*, Vol. 109, Art. No. 108063.
14. Prakash, A.J. and Ari, S. (2019) 'AAMI Standard Cardiac Arrhythmia Detection with Random Forest Using Mixed Features', in *Proceedings of the IEEE 16th India Council International Conference (INDICON)*, pp. 1–4.
15. Qiu, J. et al. (2016) 'Going Deeper with Embedded FPGA Platform for Convolutional Neural Network', in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pp. 26–35.
16. Ran, S., Yang, X., Liu, M., Zhang, Y., Cheng, C., Zhu, H. and Yuan, Y. (2022) 'Homecare Oriented ECG Diagnosis with Large-Scale Deep Neural Network for

Continuous Monitoring on Embedded Devices', *IEEE Transactions on Instrumentation and Measurement*, Vol. 71, pp. 1–13.

17. Sreenivasulu, T.M. and Meenpal, T. (2019) 'Efficient Hardware Implementation of 2D Convolution on FPGA for Image Processing Application', in *Proceedings of the IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)*.
18. Véstias, M.P., Duarte, R.P., de Sousa, J.T. and Neto, H.C. (2019) 'Fast Convolutional Neural Networks in Low Density FPGAs Using Zero-Skipping and Weight Pruning', *Electronics*, Vol. 8, No. 11, Art. No. 1321.
19. Wei, Y. et al. (2020) 'A Review of Algorithm & Hardware Design for AI-Based Biomedical Applications', *IEEE Transactions on Biomedical Circuits and Systems*, Vol. 14, No. 2, pp. 145–163.
20. Wong, D.L.T., Li, Y., John, D., Ho, W.K. and Heng, C.-H. (2022) 'An Energy Efficient ECG Ventricular Ectopic Beat Classifier Using Binarized CNN for Edge AI Devices', *IEEE Transactions on Biomedical Circuits and Systems*, Vol. 16, No. 2, pp. 222–232.
21. Wu, N., Li, Y., Yang, H., Chen, H., Dai, S., Hao, C., Yu, C. and Xie, Y. (2024) 'Survey of Machine Learning for Software-Assisted Hardware Design Verification: Past, Present, and Prospect', *ACM Transactions on Design Automation of Electronic Systems*, Vol. 29, No. 4.
22. Zhang, Q., Cao, J., Zhang, Y., Zhang, S., Zhang, Q. and Yu, D. (2019) 'FPGA Implementation of Quantized Convolutional Neural Networks', in *Proceedings of the IEEE 19th International Conference on Communication Technology (ICCT)*, pp. 1605–1610.